

Short Questions

1. What is CSS Flexbox?

A CSS layout model for arranging elements in a **one-dimensional space** (row or column). It helps align, distribute, and order items inside a container using properties like display: flex, justify-content, align-items, and flex-direction.

2. What is a favicon in HTML?

A small icon shown in the browser tab, bookmarks, or history. Linked using:

```
<link rel="icon" href="favicon.ico">
```

3. What is responsive design?

A design approach that makes web pages **adapt to different screen sizes** (mobile, tablet, desktop) using flexible grids, media queries, and responsive elements.

4. What is JSX in React?

A syntax extension for JavaScript that allows writing **HTML-like code inside JS**. React converts JSX to React.createElement() calls.

5. What is an external stylesheet?

A separate .css file linked to an HTML file using <link> tag. Keeps **styles separate** from HTML.

Example:

```
<link rel="stylesheet" href="style.css">
```

6. What is JavaScript event bubbling?

When an event triggers on a child element and then **propagates up** to its parent elements. Can be stopped with event.stopPropagation().

7. Define Bootstrap Card.

A flexible **container component** that can include header, body, footer, image, and text. Used for displaying content in a card layout.

Example:

```
<div class="card">

  

  <div class="card-body">

    <h5 class="card-title">Title</h5>

    <p class="card-text">Description</p>
```

</div>

</div>

8. What is the use of meta viewport tag?

Ensures **mobile responsiveness** by controlling viewport width and zoom:

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

9. What are HTML Semantic Tags?

Tags that **describe the meaning** of content, like <header>, <nav>, <main>, <article>, <section>, <footer>.

10. What is the difference between CSS Class and ID?

- **Class (.classname):** reusable, multiple elements can use it.
- **ID (#idname):** unique, only one element per page.
- Specificity: ID > Class in CSS priority.

11. What is a Bootstrap Container?

A wrapper for **responsive layouts**. Types:

- .container → fixed-width, responsive breakpoints.
- .container-fluid → full-width.

12. What is a React Component?

A reusable **UI building block** in React (function or class) that returns JSX and may manage state or props.

13. What is an Anchor Tag in HTML?

<a> tag used for **hyperlinks** to other pages, sections, or external URLs:

```
<a href="https://example.com">Link</a>
```

Long Questions

1. Difference between Flexbox and CSS Grid

Feature	Flexbox	Grid
Dimension	1D (row OR column)	2D (rows AND columns)

Feature	Flexbox	Grid
Use	Align items, navbars, lists	Full-page layouts, galleries, dashboards
Example	display: flex; justify-content: space-between;	display: grid; grid-template-columns: 1fr 2fr;

Flexbox example:

```
<div style="display:flex; justify-content:space-between;">
  <div>Item 1</div>
  <div>Item 2</div>
</div>
```

Grid example:

```
<div style="display:grid; grid-template-columns: 100px 1fr 100px; gap:10px;">
  <div>Left</div><div>Center</div><div>Right</div>
</div>
```

2. Validate a form using JavaScript

```
<form id="signup">
  <input id="email" type="email" placeholder="Email">
  <input id="password" type="password" placeholder="Password">
  <button type="submit">Submit</button>
  <div id="errors"></div>
</form>
```

```
<script>
const form = document.getElementById('signup');
form.addEventListener('submit', (e) => {
```

```
e.preventDefault();

const email = form.email.value.trim();

const password = form.password.value;

let errors = [];

if(!email) errors.push("Email required");

if(password.length < 6) errors.push("Password min 6 chars");

document.getElementById('errors').innerHTML = errors.join("<br>");

});

</script>
```

3. React Routing (BrowserRouter, Routes, Link)

- BrowserRouter → wraps app, handles URL.
- Routes → contains multiple Route components.
- Link → navigation without page reload.

Example:

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

<BrowserRouter>

  <nav><Link to="/">Home</Link></nav>

  <Routes>

    <Route path="/" element={<Home />} />

    <Route path="/about" element={<About />} />

  </Routes>

</BrowserRouter>
```

4. Full HTML5 structure

```
<!DOCTYPE html>
```

```
<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Page Title</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

  <header>Header</header>

  <main>Main content</main>

  <footer>Footer</footer>

  <script src="app.js"></script>

</body>

</html>
```

5. JS program: Change background color every 3 seconds

```
<script>

const colors = ['#ffadad','#ffd6a5','#fdffb6','#caffbf','#9bf6ff','#a0c4ff','#bdb2ff'];

let i = 0;

setInterval(() => {

  document.body.style.backgroundColor = colors[i % colors.length];

  i++;

}, 3000);

</script>
```

6. CSS Box Model

- **Content** → actual content
- **Padding** → space inside border
- **Border** → border line
- **Margin** → space outside border

```
div {  
  width: 200px;  
  padding: 10px;  
  border: 5px solid black;  
  margin: 20px;  
}
```

7. Steps to build responsive layout using Bootstrap

1. Include Bootstrap CSS/JS via CDN.
 2. Use .container or .container-fluid.
 3. Use .row and .col-* classes for columns.
 4. Use responsive utilities (.d-none, .text-center).
 5. Test across breakpoints.
-

8. Install React & first functional component

```
npx create-react-app my-app
```

```
cd my-app
```

```
npm start
```

Component Example:

```
function Hello(){ return <h1>Hello, World!</h1>; }  
export default Hello;
```

9. Digital timetable with time & date

```
<div id="clock"></div>

<script>

function updateClock(){

  const now = new Date();

  document.getElementById('clock').textContent = now.toLocaleString();

}

setInterval(updateClock, 1000);

</script>
```

10. Synchronous vs Asynchronous Web & Google rendering

- **Synchronous:** tasks block execution until finished.
- **Asynchronous:** tasks run without blocking (AJAX, fetch, setTimeout).

Google rendering steps:

1. DNS lookup & fetch HTML.
2. Parse HTML → build DOM.
3. Parse CSS → build CSSOM.
4. JS execution (blocking if not defer/async).
5. Construct render tree → layout → paint → display.
6. Index page content after rendering.